



**PixelLang: un lenguaje para describir la máquina de estados
de los personajes de un videojuego 2D**

Trabajo parcial

Código	Apellidos y nombres
U20221G222	Ruiz Soto, Diego Gilmer
U202417368	Ramos Parihuaman, David Miguel
U20241D937	Quito Ancasi, Antony Rodrigo
U202412516	Monge Jiménez, Mateo Alonso

Universidad Peruana de Ciencias Aplicadas

Teoría de Compiladores, sección 13962

Docente: Marco Antonio Sobrevilla Cabezudo

Grupo 5

Octubre de 2026

PixelLang: un lenguaje para describir la máquina de estados de los personajes de un videojuego 2D

Problemática y motivación

En un videojuego 2D, el comportamiento de cada personaje se describe con una máquina de estados finitos. El personaje está en reposo, saltando, atacando o aturdido, y cambia de estado cuando ocurre un evento, como pulsar un botón o recibir un golpe (Nystrom, 2014). Sin embargo, esa máquina casi nunca se escribe como tal. En el código del juego queda repartida en condicionales que revisan banderas como *saltando* o *atacando*, y nada impide que dos banderas incompatibles queden activas a la vez (Nystrom, 2014). En los editores visuales de motores como Unity, los estados se dibujan como nodos, pero las condiciones de cada transición quedan en paneles separados (Unity Technologies, s. f.).

El problema es que nada revisa esa máquina antes de jugar. Un estado al que nunca se llega, una transición hacia un estado que ya no existe o un estado del que no se puede salir solo se descubren durante el juego, y los diagramas de estados planos se vuelven difíciles de mantener a medida que crecen (Harel, 1987). Encontrar estos errores toma tiempo, porque las pruebas de videojuegos dependen casi solo de partidas manuales (Politowski et al., 2021) y los equipos evitan automatizarlas, ya que quedan obsoletas cada vez que cambia el diseño (Murphy-Hill et al., 2014).

Existen formatos como SCXML (World Wide Web Consortium, 2015) y librerías como XState (Stately, s. f.) que describen máquinas de estados en texto, pero son de propósito general y están pensados para integrarse en intérpretes o en aplicaciones web. Por eso proponemos PixelLang, un lenguaje de dominio específico en el que la máquina de estados de cada personaje se escribe como texto. Un lenguaje de este tipo permite analizar y verificar los programas en los términos del dominio (Mernik et al., 2005). En PixelLang, cada personaje es un autómata finito $A = \{Q, \Sigma, \delta, q_0, F\}$, de modo que el compilador puede comprobar, antes de ejecutar nada, que ese autómata esté bien formado.

Objetivos

El objetivo general es desarrollar un compilador para PixelLang que verifique, al compilar, que la máquina de estados de cada personaje esté bien formada, y que la traduzca a código ejecutable. Los objetivos específicos son los siguientes:

1. Definir el léxico del lenguaje e implementar el analizador léxico con expresiones regulares en ANTLR4.
2. Diseñar una gramática libre de contexto para cada construcción, validarla con derivaciones más a la izquierda e implementar el analizador sintáctico en ANTLR4.
3. Construir la tabla de símbolos e implementar un analizador semántico que detecte los errores del autómata.
4. Generar código ejecutable con LLVM (Lattner y Adve, 2004).

Construcciones del lenguaje

Un programa de PixelLang declara uno o más personajes y, al final, cero o más simulaciones. Cada personaje declara sus atributos, sus eventos y sus estados, y cada estado lista las transiciones que salen de él. La Tabla 1 resume las siete construcciones del lenguaje con un ejemplo de cada una, y el Apéndice A contiene los cinco ejemplos de cada construcción.

Tabla 1

Construcciones de PixelLang

Construcción	Descripción	Ejemplo
Programa y personaje	Lista de personajes seguida de simulaciones. Un personaje contiene sus atributos, sus eventos y al menos un estado.	<pre> personaje Puerta { eventos ser_abierta; inicial estado Cerrada { cuando ser_abierta -> Abierta; } final estado Abierta { } } simular Puerta: ser_abierta; </pre>
Declaración de atributos	Variable del personaje con nombre, tipo y valor inicial.	<pre> vida: entero = 100; </pre>
Declaración de eventos	Lista de los eventos del personaje, que forman su alfabeto.	<pre> eventos saltar, atacar; </pre>

Construcción	Descripción	Ejemplo
Declaración de estados	Estado con un modificador opcional (<i>inicial</i> o <i>final</i>), acciones al entrar y transiciones.	estado Saltando { al_entrar { y = y + 2.5; } cuando aterrizar -> Reposo; cuando recibir_golpe -> Aturdido; }
Transiciones	Evento que la dispara, guarda opcional con <i>si</i> y estado destino.	cuando fin_aturdimiento si vida <= 0 -> Derrotado;
Acciones	Asignación a un atributo y selección <i>si</i> con <i>sino</i> opcional.	si vida < 50 { velocidad = 1.0; }
Expresiones	Operaciones lógicas, relacionales y aritméticas con precedencia.	municion > 0 && escudo != verdadero

Analizador léxico

La Tabla 2 presenta el léxico de PixelLang en la forma token: lista de lexemas. El analizador léxico está implementado con expresiones regulares en ANTLR4 (Parr, 2013). Si dos reglas reconocen un lexema de la misma longitud, gana la que aparece primero. Por eso las palabras reservadas se definen antes que ID. Además, siempre gana el lexema más largo, así que *siguiente* es un identificador y no la palabra *si* seguida de otras letras.

Tabla 2

Léxico de PixelLang en la forma token: lista de lexemas

Categoría	Token: lista de lexemas
Palabras reservadas	PERSONAJE: personaje EVENTOS: eventos ESTADO: estado INICIAL: inicial FINAL: final AL_ENTRAR: al_entrar CUANDO: cuando SI: si SINO: sino SIMULAR: simular
Tipos y valores lógicos	ENTERO: entero REAL: real LOGICO: logico, lógico VERDADERO: verdadero FALSO: falso
Operadores	FLECHA: -> O_LOGICO: Y_LOGICO: && NO_LOGICO: ! IGUAL: == DISTINTO: != MENOR_IGUAL: <= MAYOR_IGUAL: >= MENOR: < MAYOR: > ASIGNA: = MAS: + MENOS: - POR: * ENTRE: /
Signos de puntuación	PAR_A: (PAR_C:) LLAVE_A: { LLAVE_C: } DOS_PUNTOS: : PUNTO_COMA: ; COMA: ,
Números e identificadores	NUM_ENTERO: [0-9]+ (por ejemplo, 0, 25, 100) NUM_REAL: [0-9]+ '.' [0-9]+ (por ejemplo, 0.0, 2.5, 9.8) ID: LETRA (LETRA DIGITO)* (por ejemplo, vida, y, Caballero, daño_golpe)
Se descartan	COMENTARIO_LINEA: '/'/' ~[\r\n]* COMENTARIO_BLOQUE: '/' '*' .*? '*' '/' ESPACIOS: [\t\r\n]+

Nota. Los lexemas van en negrita.

Analizador sintáctico

La Tabla 3 contiene las gramáticas libres de contexto de las siete construcciones, escritas con la notación vista en clase (Aho et al., 2006): los no terminales van en cursiva, los terminales en negrita, las alternativas se separan con | y ϵ es la cadena vacía. Los terminales **id**, **num** y **dec** representan los tokens ID, NUM_ENTERO y NUM_REAL. La gramática de expresiones tiene un no terminal por cada nivel de precedencia, por lo que no es ambigua.

Tabla 3

Gramáticas libres de contexto de PixelLang

Gramáticas G1 a G4	Gramáticas G5 a G7
G1. Programa y personaje <i>programa</i> → <i>lista_pers lista_sims</i> <i>lista_pers</i> → <i>lista_pers decl_pers</i> <i>decl_pers</i> <i>lista_sims</i> → <i>lista_sims simulacion</i> ϵ <i>decl_pers</i> → personaje id { <i>lista_atrs decl_evs</i> <i>lista_est</i> } <i>lista_atrs</i> → <i>lista_atrs decl_atr</i> ϵ <i>lista_est</i> → <i>lista_est decl_est</i> <i>decl_est</i> <i>simulacion</i> → simular id : <i>lista_ids</i> ; G2. Atributos <i>decl_atr</i> → id : <i>tipo = expr</i> ; <i>tipo</i> → entero real logico G3. Eventos <i>decl_evs</i> → eventos <i>lista_ids</i> ; <i>lista_ids</i> → <i>lista_ids</i> , id id G4. Estados <i>decl_est</i> → modif estado id { <i>entrada lista_trs</i> } <i>modif</i> → inicial final ϵ <i>entrada</i> → al_entrar <i>bloque</i> ϵ <i>lista_trs</i> → <i>lista_trs transicion</i> ϵ	G5. Transiciones <i>transicion</i> → cuando id guarda -> id ; <i>guarda</i> → si <i>expr</i> ϵ G6. Acciones <i>sentencia</i> → id = expr ; si <i>expr bloque</i> <i>alternativa</i> <i>alternativa</i> → sino <i>bloque</i> ϵ <i>bloque</i> → { <i>lista_sent</i> } <i>lista_sent</i> → <i>lista_sent sentencia</i> ϵ G7. Expresiones <i>expr</i> → <i>expr</i> <i>conj</i> <i>conj</i> <i>conj</i> → <i>conj</i> && <i>rel</i> <i>rel</i> <i>rel</i> → <i>arit oprel arit</i> <i>arit</i> <i>oprel</i> → < > <= >= == != <i>arit</i> → <i>arit + term</i> <i>arit - term</i> <i>term</i> <i>term</i> → <i>term * unario</i> <i>term / unario</i> <i>unario</i> <i>unario</i> → ! <i>unario</i> - <i>unario</i> <i>factor</i> <i>factor</i> → (<i>expr</i>) num dec verdadero falso id

Para validar cada gramática derivamos por la izquierda cuatro de sus ejemplos. Las 28 derivaciones están en el Apéndice B y la Tabla 4 muestra una de ellas. Cuando el no terminal más a la izquierda pertenece a otra gramática, su subderivación se resume en un solo paso con $\Rightarrow^*$.

Tabla 4*Derivación más a la izquierda del ejemplo 5.2*

Paso	Forma sentencial	Producción aplicada
0	<i>transicion</i>	<i>transicion</i> → cuando id guarda -> id ;
1	cuando id guarda -> id ;	<i>guarda</i> → si expr
2	cuando id si expr -> id ;	<i>expr</i> ⇒* id <= num (G7)
3	cuando id si id <= num -> id ;	<i>fin</i>

Nota. Ejemplo 5.2: cuando `fin_aturdimiento si vida <= 0 -> Derrotado;`. El paso 2 resume la subderivación de *expr* en la gramática G7.

Analizador semántico

El analizador semántico es un *visitor* de ANTLR4 que recorre el árbol sintáctico y construye la tabla de símbolos. La tabla tiene dos ámbitos: el global, con los personajes, y el de cada personaje, con sus atributos y tipos, sus eventos y sus estados. Con esa tabla verifica, entre otros, los dos errores de la Tabla 5, que corresponden a la definición de autómata finito. La función de transición δ solo puede llevar a estados de Q, y para cada estado y evento hay un único destino.

Tabla 5*Errores semánticos principales*

Código	Error	Regla que viola	Ejemplo
E01	Transición hacia un estado no declarado	δ solo puede llevar a estados de Q	estado Saltando { cuando atacar -> Volando; }
E03	Transiciones no deterministas	$\delta(q, e)$ tiene un único destino	cuando saltar -> Saltando; cuando saltar -> Reposo;

Nota. En el ejemplo de E01, Volando no es un estado del personaje. En el de E03, la segunda transición nunca se toma.

Además de estos dos errores, el compilador verifica otros ocho, descritos en el Apéndice C. Cada error se informa con su código, su línea y su columna, y el análisis continúa para reportar todos los errores juntos. Las pruebas de los tres analizadores, con entradas válidas e inválidas, están en el Apéndice D.

Referencias

- Aho, A. V., Lam, M. S., Sethi, R. y Ullman, J. D. (2006). *Compilers: Principles, techniques, and tools* (2.^a ed.). Pearson.
- Harel, D. (1987). Statecharts: A visual formalism for complex systems. *Science of Computer Programming*, 8(3), 231–274. [https://doi.org/10.1016/0167-6423\(87\)90035-9](https://doi.org/10.1016/0167-6423(87)90035-9)
- Lattner, C. y Adve, V. (2004). LLVM: A compilation framework for lifelong program analysis & transformation. En *International Symposium on Code Generation and Optimization, 2004. CGO 2004* (pp. 75–86). IEEE.
<https://doi.org/10.1109/CGO.2004.1281665>
- Mernik, M., Heering, J. y Sloane, A. M. (2005). When and how to develop domain-specific languages. *ACM Computing Surveys*, 37(4), 316–344.
<https://doi.org/10.1145/1118890.1118892>
- Murphy-Hill, E., Zimmermann, T. y Nagappan, N. (2014). Cowboys, ankle sprains, and keepers of quality: How is video game development different from software development? En *Proceedings of the 36th International Conference on Software Engineering* (pp. 1–11). ACM. <https://doi.org/10.1145/2568225.2568226>
- Nystrom, R. (2014). State. En *Game programming patterns*. Genever Benning.
<https://gameprogrammingpatterns.com/state.html>
- Parr, T. (2013). *The definitive ANTLR 4 reference*. The Pragmatic Bookshelf.
- Politowski, C., Petrillo, F. y Guéhéneuc, Y.-G. (2021). *A survey of video game testing* [Preprint]. arXiv. <https://doi.org/10.48550/arXiv.2103.06431>
- Stately. (s. f.). *XState*. Recuperado el 4 de octubre de 2026, de <https://stately.ai/docs/xstate>
- Unity Technologies. (s. f.). *Introduction to Animator Controllers*. Recuperado el 4 de octubre de 2026, de <https://docs.unity3d.com/Manual/class-AnimatorController.html>
- World Wide Web Consortium. (2015). *State chart XML (SCXML): State machine notation for control abstraction* (W3C Recommendation). <https://www.w3.org/TR/scxml/>

Apéndice A

Ejemplos de las construcciones

Las Tablas A1 a A7 contienen los cinco ejemplos de cada construcción. Todos son aceptados por el analizador sintáctico a partir de la regla de su construcción, y su número se usa en las derivaciones del Apéndice B.

Tabla A1

Ejemplos de la construcción programa y declaración de personaje

Ejemplo	Código
1.1	<pre> personaje Estatua { eventos ser_tocada; inicial estado Quieta { cuando ser_tocada -> Quieta; } }</pre>
1.2	<pre> personaje Puerta { eventos ser_abierta; inicial estado Cerrada { cuando ser_abierta -> Abierta; } final estado Abierta { } } simular Puerta: ser_abierta;</pre>
1.3	<pre> personaje Moneda { valor: entero = 10; eventos ser_recogida; inicial estado Visible { cuando ser_recogida -> Recogida; } final estado Recogida { } }</pre>
1.4	<pre> personaje Boton { eventos ser_pulsado; inicial estado Suelto { cuando ser_pulsado -> Suelto; } } personaje Bandera { eventos ser_tocada; inicial estado Abajo { cuando ser_tocada -> Arriba; } final estado Arriba { } }</pre>

Ejemplo	Código
1.5	<pre> personaje Torreta { municion: entero = 3; eventos disparar, fin_disparo, fin_recarga; inicial estado Lista { cuando disparar si municion > 0 -> Disparando; cuando disparar -> Recargando; } estado Disparando { al_entrar { municion = municion - 1; } cuando fin_disparo -> Lista; } estado Recargando { al_entrar { municion = 3; } cuando fin_recarga -> Lista; } } simular Torreta: disparar, fin_disparo, disparar; </pre>

Tabla A2*Ejemplos de la construcción declaración de atributos*

Ejemplo	Código
2.1	vida: entero = 100;
2.2	velocidad: real = 2.5;
2.3	vivo: logico = verdadero;
2.4	gravedad: real = -9.8;
2.5	daño_critico: entero = daño_golpe * 2;

Tabla A3*Ejemplos de la construcción declaración de eventos*

Ejemplo	Código
3.1	eventos saltar;
3.2	eventos saltar, atacar;
3.3	eventos saltar, atacar, recibir_golpe;
3.4	eventos ver_jugador, perder_jugador, recibir_golpe;
3.5	eventos pulsar_a, pulsar_b, soltar_a, soltar_b, pausar;

Tabla A4*Ejemplos de la construcción declaración de estados*

Ejemplo	Código
4.1	estado Reposo { }

Ejemplo	Código
4.2	inicial estado Reposo { cuando saltar -> Saltando; }
4.3	final estado Derrotado { }
4.4	estado Aturdido { al_entrar { vida = vida - daño_golpe; } cuando recibir_golpe -> Aturdido; }
4.5	estado Saltando { al_entrar { y = y + 2.5; } cuando aterrizar -> Reposo; cuando recibir_golpe -> Aturdido; }

Tabla A5*Ejemplos de la construcción transiciones*

Ejemplo	Código
5.1	cuando saltar -> Saltando;
5.2	cuando fin_aturdimiento si vida <= 0 -> Derrotado;
5.3	cuando fin_ataque -> Reposo;
5.4	cuando disparar si municion > 0 && !bloqueada -> Disparando;
5.5	cuando recibir_golpe si escudo vida > 50 -> Bloqueando;

Tabla A6*Ejemplos de la construcción acciones: asignación y selección*

Ejemplo	Código
6.1	vida = 100;
6.2	velocidad = velocidad * 0.5;
6.3	si vida < 50 { velocidad = 1.0; }
6.4	si vida < 50 { velocidad = 1.0; } sino { velocidad = 1.5; }
6.5	si escudo { escudo = falso; } sino { vida = vida - daño_golpe; si vida <= 0 { vivo = falso; } }

Tabla A7*Ejemplos de la construcción expresiones*

Ejemplo	Código
7.1	vida - 25 < 10

Ejemplo	Código
7.2	<code>2 * (velocidad + 1.5) / 4</code>
7.3	<code>vida <= 0 !vivo</code>
7.4	<code>municion > 0 && escudo != verdadero</code>
7.5	<code>-gravedad * tiempo >= 9.8</code>

Apéndice B

Derivaciones más a la izquierda

Para cada gramática derivamos cuatro de los cinco ejemplos de su construcción, elegidos para que entre los cuatro se use cada producción de la gramática. La excepción es la gramática de expresiones, con 27 producciones: quedan fuera $\geq$, $=$, el cambio de signo y **falso**, que se derivan igual que $>$, $!=$, la negación y **verdadero**.

En cada paso se reemplaza el no terminal más a la izquierda, y la última columna indica la producción aplicada para obtener la fila siguiente. Cuando ese no terminal pertenece a otra gramática, su subderivación se resume en un paso con $\Rightarrow^*$, y entre paréntesis se indica la gramática a la que pertenece. Cada derivación termina en la cadena de tokens del ejemplo.

Tabla B1

Derivaciones más a la izquierda de la gramática G1 (programa y declaración de personaje)

Paso	Forma sentencial	Producción aplicada
Ejemplo 1.1 (código en el Apéndice A)		
Cadena de tokens: personaje id { eventos id ; inicial estado id { cuando id -> id ; } }		
0	<i>programa</i>	<i>programa</i> $\rightarrow$ <i>lista_pers lista_sims</i>
1	<i>lista_pers lista_sims</i>	<i>lista_pers</i> $\rightarrow$ <i>decl_pers</i>
2	<i>decl_pers lista_sims</i>	<i>decl_pers</i> $\rightarrow$ personaje id { lista_atrs decl_evs lista_est }
3	personaje id { lista_atrs decl_evs lista_est } <i>lista_sims</i>	<i>lista_atrs</i> $\rightarrow$ ϵ
4	personaje id { decl_evs lista_est } <i>lista_sims</i>	<i>decl_evs</i> $\Rightarrow^*$ eventos id ; (G3)
5	personaje id { eventos id ; lista_est } <i>lista_sims</i>	<i>lista_est</i> $\rightarrow$ <i>decl_est</i>
6	personaje id { eventos id ; decl_est } <i>lista_sims</i>	<i>decl_est</i> $\Rightarrow^*$ inicial estado id { cuando id -> id ; } (G4)
7	personaje id { eventos id ; inicial estado id { cuando id -> id ; } } <i>lista_sims</i>	<i>lista_sims</i> $\rightarrow$ ϵ
8	personaje id { eventos id ; inicial estado id { cuando id -> id ; } }	<i>fin</i>

Ejemplo 1.2 (código en el Apéndice A)

Cadena de tokens: **personaje id { eventos id ; inicial estado id { cuando id -> id ; } final estado id { } } simular id : id ;**

0	<i>programa</i>	<i>programa</i> $\rightarrow$ <i>lista_pers lista_sims</i>
1	<i>lista_pers lista_sims</i>	<i>lista_pers</i> $\rightarrow$ <i>decl_pers</i>
2	<i>decl_pers lista_sims</i>	<i>decl_pers</i> $\rightarrow$ personaje id { lista_atrs decl_evs lista_est }
3	personaje id { lista_atrs decl_evs lista_est } <i>lista_sims</i>	<i>lista_atrs</i> $\rightarrow$ ϵ
4	personaje id { decl_evs lista_est } <i>lista_sims</i>	<i>decl_evs</i> $\Rightarrow^*$ eventos id ; (G3)
5	personaje id { eventos id ; lista_est } <i>lista_sims</i>	<i>lista_est</i> $\rightarrow$ <i>lista_est decl_est</i>
6	personaje id { eventos id ; lista_est decl_est } <i>lista_sims</i>	<i>lista_est</i> $\rightarrow$ <i>decl_est</i>

Paso	Forma sentencial	Producción aplicada
7	personaje id { eventos id ; decl_est decl_est } lista_sims	$decl_est \Rightarrow^* \text{inicial estado id \{ cuando id -> id ; \}} (G4)$
8	personaje id { eventos id ; inicial estado id { cuando id -> id ; } decl_est } lista_sims	$decl_est \Rightarrow^* \text{final estado id \{ \}} (G4)$
9	personaje id { eventos id ; inicial estado id { cuando id -> id ; } final estado id { \} } lista_sims	$lista_sims \rightarrow lista_sims \text{ simulacion}$
10	personaje id { eventos id ; inicial estado id { cuando id -> id ; } final estado id { \} } lista_sims simulacion	$lista_sims \rightarrow \epsilon$
11	personaje id { eventos id ; inicial estado id { cuando id -> id ; } final estado id { \} } simulacion	$simulacion \rightarrow \text{simular id : lista_ids ;}$
12	personaje id { eventos id ; inicial estado id { cuando id -> id ; } final estado id { \} } simular id : lista_ids ;	$lista_ids \Rightarrow^* \text{id} (G3)$
13	personaje id { eventos id ; inicial estado id { cuando id -> id ; } final estado id { \} } simular id : id ;	fin

Ejemplo 1.3 (código en el Apéndice A)

Cadena de tokens: **personaje id { id : entero = num ; eventos id ; inicial estado id { cuando id -> id ; } final estado id { \} }**

0	<i>programa</i>	$programa \rightarrow lista_pers lista_sims$
1	<i>lista_pers lista_sims</i>	$lista_pers \rightarrow decl_pers$
2	<i>decl_pers lista_sims</i>	$decl_pers \rightarrow \text{personaje id \{ lista_attrs decl_evs lista_est \}}$
3	personaje id { lista_attr decl_evs lista_est } lista_sims	$lista_attrs \rightarrow lista_attrs decl_atr$
4	personaje id { lista_attr decl_atr decl_evs lista_est } lista_sims	$lista_attrs \rightarrow \epsilon$
5	personaje id { decl_atr decl_evs lista_est } lista_sims	$decl_atr \Rightarrow^* \text{id : entero = num ;} (G2)$
6	personaje id { id : entero = num ; decl_evs lista_est } lista_sims	$decl_evs \Rightarrow^* \text{eventos id ;} (G3)$
7	personaje id { id : entero = num ; eventos id ; lista_est } lista_sims	$lista_est \rightarrow lista_est decl_est$
8	personaje id { id : entero = num ; eventos id ; lista_est decl_est } lista_sims	$lista_est \rightarrow decl_est$
9	personaje id { id : entero = num ; eventos id ; decl_est decl_est } lista_sims	$decl_est \Rightarrow^* \text{inicial estado id \{ cuando id -> id ; \}} (G4)$
10	personaje id { id : entero = num ; eventos id ; inicial estado id { cuando id -> id ; } decl_est } lista_sims	$decl_est \Rightarrow^* \text{final estado id \{ \}} (G4)$
11	personaje id { id : entero = num ; eventos id ; inicial estado id { cuando id -> id ; } final estado id { \} } lista_sims	$lista_sims \rightarrow \epsilon$
12	personaje id { id : entero = num ; eventos id ; inicial estado id { cuando id -> id ; } final estado id { \} }	fin

Ejemplo 1.4 (código en el Apéndice A)

Cadena de tokens: **personaje id { eventos id ; inicial estado id { cuando id -> id ; } } personaje id { eventos id ; inicial estado id { cuando id -> id ; } final estado id { \} }**

0	<i>programa</i>	$programa \rightarrow lista_pers lista_sims$
1	<i>lista_pers lista_sims</i>	$lista_pers \rightarrow lista_pers decl_pers$
2	<i>lista_pers decl_pers lista_sims</i>	$lista_pers \rightarrow decl_pers$
3	<i>decl_pers decl_pers lista_sims</i>	$decl_pers \rightarrow \text{personaje id \{ lista_attrs decl_evs lista_est \}}$
4	personaje id { lista_attr decl_evs lista_est } decl_pers lista_sims	$lista_attrs \rightarrow \epsilon$

Paso	Forma sentencial	Producción aplicada
5	personaje id { decl_evs lista_est } decl_pers lista_sims	$decl_evs \Rightarrow^* \text{eventos id ; (G3)}$
6	personaje id { eventos id ; lista_est } decl_pers lista_sims	$lista_est \rightarrow decl_est$
7	personaje id { eventos id ; decl_est } decl_pers lista_sims	$decl_est \Rightarrow^* \text{inicial estado id { cuando id -> id ; } (G4)}$
8	personaje id { eventos id ; inicial estado id { cuando id -> id ; } } decl_pers lista_sims	$decl_pers \rightarrow \text{personaje id { lista_atrs decl_evs lista_est } }$
9	personaje id { eventos id ; inicial estado id { cuando id -> id ; } } personaje id { lista_atrs decl_evs lista_est } lista_sims	$lista_atrs \rightarrow \epsilon$
10	personaje id { eventos id ; inicial estado id { cuando id -> id ; } } personaje id { decl_evs lista_est } lista_sims	$decl_evs \Rightarrow^* \text{eventos id ; (G3)}$
11	personaje id { eventos id ; inicial estado id { cuando id -> id ; } } personaje id { eventos id ; lista_est } lista_sims	$lista_est \rightarrow lista_est decl_est$
12	personaje id { eventos id ; inicial estado id { cuando id -> id ; } } personaje id { eventos id ; lista_est decl_est } lista_sims	$lista_est \rightarrow decl_est$
13	personaje id { eventos id ; inicial estado id { cuando id -> id ; } } personaje id { eventos id ; decl_est decl_est } lista_sims	$decl_est \Rightarrow^* \text{inicial estado id { cuando id -> id ; } (G4)}$
14	personaje id { eventos id ; inicial estado id { cuando id -> id ; } } personaje id { eventos id ; inicial estado id { cuando id -> id ; } decl_est } lista_sims	$decl_est \Rightarrow^* \text{final estado id { } (G4)}$
15	personaje id { eventos id ; inicial estado id { cuando id -> id ; } } personaje id { eventos id ; inicial estado id { cuando id -> id ; } final estado id { } } lista_sims	$lista_sims \rightarrow \epsilon$
16	personaje id { eventos id ; inicial estado id { cuando id -> id ; } } personaje id { eventos id ; inicial estado id { cuando id -> id ; } final estado id { } }	fin

Tabla B2

Derivaciones más a la izquierda de la gramática G2 (declaración de atributos)

Paso	Forma sentencial	Producción aplicada
Ejemplo 2.1: vida: entero = 100;		
Cadena de tokens: id : entero = num ;		
0	decl_atr	$decl_atr \rightarrow \text{id : tipo = expr ;}$
1	id : tipo = expr ;	$tipo \rightarrow \text{entero}$
2	id : entero = expr ;	$expr \Rightarrow^* \text{num (G7)}$
3	id : entero = num ;	fin
Ejemplo 2.2: velocidad: real = 2.5;		
Cadena de tokens: id : real = dec ;		
0	decl_atr	$decl_atr \rightarrow \text{id : tipo = expr ;}$
1	id : tipo = expr ;	$tipo \rightarrow \text{real}$
2	id : real = expr ;	$expr \Rightarrow^* \text{dec (G7)}$
3	id : real = dec ;	fin
Ejemplo 2.3: vivo: logico = verdadero;		
Cadena de tokens: id : logico = verdadero ;		
0	decl_atr	$decl_atr \rightarrow \text{id : tipo = expr ;}$
1	id : tipo = expr ;	$tipo \rightarrow \text{logico}$

Paso	Forma sentencial	Producción aplicada
2	id : logico = <i>expr</i> ;	<i>expr</i> ⇒* verdadero (G7)
3	id : logico = verdadero ;	<i>fin</i>

Ejemplo 2.4: gravedad: real = -9.8;
Cadena de tokens: **id : real = - dec** ;

0	<i>decl_atr</i>	<i>decl_atr</i> → id : <i>tipo</i> = <i>expr</i> ;
1	id : <i>tipo</i> = <i>expr</i> ;	<i>tipo</i> → real
2	id : real = <i>expr</i> ;	<i>expr</i> ⇒* - dec (G7)
3	id : real = - dec ;	<i>fin</i>

Tabla B3

Derivaciones más a la izquierda de la gramática G3 (declaración de eventos)

Paso	Forma sentencial	Producción aplicada
------	------------------	---------------------

Ejemplo 3.1: eventos saltar;
Cadena de tokens: **eventos id** ;

0	<i>decl_evs</i>	<i>decl_evs</i> → eventos <i>lista_ids</i> ;
1	eventos <i>lista_ids</i> ;	<i>lista_ids</i> → id
2	eventos id ;	<i>fin</i>

Ejemplo 3.2: eventos saltar, atacar;
Cadena de tokens: **eventos id , id** ;

0	<i>decl_evs</i>	<i>decl_evs</i> → eventos <i>lista_ids</i> ;
1	eventos <i>lista_ids</i> ;	<i>lista_ids</i> → <i>lista_ids</i> , id
2	eventos <i>lista_ids</i> , id ;	<i>lista_ids</i> → id
3	eventos id , id ;	<i>fin</i>

Ejemplo 3.3: eventos saltar, atacar, recibir_golpe;
Cadena de tokens: **eventos id , id , id** ;

0	<i>decl_evs</i>	<i>decl_evs</i> → eventos <i>lista_ids</i> ;
1	eventos <i>lista_ids</i> ;	<i>lista_ids</i> → <i>lista_ids</i> , id
2	eventos <i>lista_ids</i> , id ;	<i>lista_ids</i> → <i>lista_ids</i> , id
3	eventos <i>lista_ids</i> , id , id ;	<i>lista_ids</i> → id
4	eventos id , id , id ;	<i>fin</i>

Ejemplo 3.5: eventos pulsar_a, pulsar_b, soltar_a, soltar_b, pausar;
Cadena de tokens: **eventos id , id , id , id , id** ;

0	<i>decl_evs</i>	<i>decl_evs</i> → eventos <i>lista_ids</i> ;
1	eventos <i>lista_ids</i> ;	<i>lista_ids</i> → <i>lista_ids</i> , id
2	eventos <i>lista_ids</i> , id ;	<i>lista_ids</i> → <i>lista_ids</i> , id
3	eventos <i>lista_ids</i> , id , id ;	<i>lista_ids</i> → <i>lista_ids</i> , id
4	eventos <i>lista_ids</i> , id , id , id ;	<i>lista_ids</i> → <i>lista_ids</i> , id
5	eventos <i>lista_ids</i> , id , id , id , id ;	<i>lista_ids</i> → id
6	eventos id , id , id , id , id ;	<i>fin</i>

Tabla B4

Derivaciones más a la izquierda de la gramática G4 (declaración de estados)

Paso	Forma sentencial	Producción aplicada
Ejemplo 4.1: estado Reposo { }		
<i>Cadena de tokens:</i> estado id { }		
0	<i>decl_est</i>	<i>decl_est</i> → <i>modif estado id { entrada lista_trs }</i>
1	<i>modif estado id { entrada lista_trs }</i>	<i>modif</i> → ϵ
2	estado id { entrada lista_trs }	<i>entrada</i> → ϵ
3	estado id { lista_trs }	<i>lista_trs</i> → ϵ
4	estado id { }	<i>fin</i>
Ejemplo 4.2: inicial estado Reposo { cuando saltar -> Saltando; }		
<i>Cadena de tokens:</i> inicial estado id { cuando id -> id ; }		
0	<i>decl_est</i>	<i>decl_est</i> → <i>modif estado id { entrada lista_trs }</i>
1	<i>modif estado id { entrada lista_trs }</i>	<i>modif</i> → inicial
2	inicial estado id { entrada lista_trs }	<i>entrada</i> → ϵ
3	inicial estado id { lista_trs }	<i>lista_trs</i> → <i>lista_trs transicion</i>
4	inicial estado id { lista_trs transicion }	<i>lista_trs</i> → ϵ
5	inicial estado id { transicion }	<i>transicion</i> ⇒* cuando id -> id ; (G5)
6	inicial estado id { cuando id -> id ; }	<i>fin</i>
Ejemplo 4.3: final estado Derrotado { }		
<i>Cadena de tokens:</i> final estado id { }		
0	<i>decl_est</i>	<i>decl_est</i> → <i>modif estado id { entrada lista_trs }</i>
1	<i>modif estado id { entrada lista_trs }</i>	<i>modif</i> → final
2	final estado id { entrada lista_trs }	<i>entrada</i> → ϵ
3	final estado id { lista_trs }	<i>lista_trs</i> → ϵ
4	final estado id { }	<i>fin</i>
Ejemplo 4.4 (código en el Apéndice A)		
<i>Cadena de tokens:</i> estado id { al_entrar { id = id - id ; } cuando id -> id ; }		
0	<i>decl_est</i>	<i>decl_est</i> → <i>modif estado id { entrada lista_trs }</i>
1	<i>modif estado id { entrada lista_trs }</i>	<i>modif</i> → ϵ
2	estado id { entrada lista_trs }	<i>entrada</i> → al_entrar bloque
3	estado id { al_entrar bloque lista_trs }	<i>bloque</i> ⇒* { id = id - id ; } (G6)
4	estado id { al_entrar { id = id - id ; } lista_trs }	<i>lista_trs</i> → <i>lista_trs transicion</i>
5	estado id { al_entrar { id = id - id ; } lista_trs transicion }	<i>lista_trs</i> → ϵ
6	estado id { al_entrar { id = id - id ; } transicion }	<i>transicion</i> ⇒* cuando id -> id ; (G5)
7	estado id { al_entrar { id = id - id ; } cuando id -> id ; }	<i>fin</i>

Tabla B5*Derivaciones más a la izquierda de la gramática G5 (transiciones)*

Paso	Forma sentencial	Producción aplicada
Ejemplo 5.1: cuando saltar -> Saltando;		
Cadena de tokens: cuando id -> id ;		
0	<i>transicion</i>	<i>transicion</i> → cuando id guarda -> id ;
1	cuando id guarda -> id ;	<i>guarda</i> → ϵ
2	cuando id -> id ;	<i>fin</i>
Ejemplo 5.2: cuando fin_aturdimiento si vida <= 0 -> Derrotado;		
Cadena de tokens: cuando id si id <= num -> id ;		
0	<i>transicion</i>	<i>transicion</i> → cuando id guarda -> id ;
1	cuando id guarda -> id ;	<i>guarda</i> → si expr
2	cuando id si expr -> id ;	<i>expr</i> ⇒* id <= num (G7)
3	cuando id si id <= num -> id ;	<i>fin</i>
Ejemplo 5.4: cuando disparar si municion > 0 && !bloqueada -> Disparando;		
Cadena de tokens: cuando id si id > num && ! id -> id ;		
0	<i>transicion</i>	<i>transicion</i> → cuando id guarda -> id ;
1	cuando id guarda -> id ;	<i>guarda</i> → si expr
2	cuando id si expr -> id ;	<i>expr</i> ⇒* id > num && ! id (G7)
3	cuando id si id > num && ! id -> id ;	<i>fin</i>
Ejemplo 5.5: cuando recibir_golpe si escudo vida > 50 -> Bloqueando;		
Cadena de tokens: cuando id si id id > num -> id ;		
0	<i>transicion</i>	<i>transicion</i> → cuando id guarda -> id ;
1	cuando id guarda -> id ;	<i>guarda</i> → si expr
2	cuando id si expr -> id ;	<i>expr</i> ⇒* id id > num (G7)
3	cuando id si id id > num -> id ;	<i>fin</i>

Tabla B6*Derivaciones más a la izquierda de la gramática G6 (acciones: asignación y selección)*

Paso	Forma sentencial	Producción aplicada
Ejemplo 6.1: vida = 100;		
Cadena de tokens: id = num ;		
0	<i>sentencia</i>	<i>sentencia</i> → id = expr ;
1	id = expr ;	<i>expr</i> ⇒* num (G7)
2	id = num ;	<i>fin</i>
Ejemplo 6.2: velocidad = velocidad * 0.5;		
Cadena de tokens: id = id * dec ;		
0	<i>sentencia</i>	<i>sentencia</i> → id = expr ;
1	id = expr ;	<i>expr</i> ⇒* id * dec (G7)
2	id = id * dec ;	<i>fin</i>

Ejemplo 6.3: si vida < 50 { velocidad = 1.0; }

Paso	Forma sentencial	Producción aplicada
<i>Cadena de tokens: si id < num { id = dec ; }</i>		
0	<i>sentencia</i>	<i>sentencia</i> → si <i>expr</i> <i>bloque</i> <i>alternativa</i>
1	si <i>expr</i> <i>bloque</i> <i>alternativa</i>	<i>expr</i> ⇒* id < num (G7)
2	si id < num <i>bloque</i> <i>alternativa</i>	<i>bloque</i> → { <i>lista_sent</i> }
3	si id < num { <i>lista_sent</i> } <i>alternativa</i>	<i>lista_sent</i> → <i>lista_sent</i> <i>sentencia</i>
4	si id < num { <i>lista_sent</i> <i>sentencia</i> } <i>alternativa</i>	<i>lista_sent</i> → ε
5	si id < num { <i>sentencia</i> } <i>alternativa</i>	<i>sentencia</i> → id = <i>expr</i> ;
6	si id < num { id = <i>expr</i> ; } <i>alternativa</i>	<i>expr</i> ⇒* dec (G7)
7	si id < num { id = dec ; } <i>alternativa</i>	<i>alternativa</i> → ε
8	si id < num { id = dec ; }	<i>fin</i>

Ejemplo 6.4: si vida < 50 { velocidad = 1.0; } sino { velocidad = 1.5; }

Cadena de tokens: si id < num { id = dec ; } sino { id = dec ; }

0	<i>sentencia</i>	<i>sentencia</i> → si <i>expr</i> <i>bloque</i> <i>alternativa</i>
1	si <i>expr</i> <i>bloque</i> <i>alternativa</i>	<i>expr</i> ⇒* id < num (G7)
2	si id < num <i>bloque</i> <i>alternativa</i>	<i>bloque</i> → { <i>lista_sent</i> }
3	si id < num { <i>lista_sent</i> } <i>alternativa</i>	<i>lista_sent</i> → <i>lista_sent</i> <i>sentencia</i>
4	si id < num { <i>lista_sent</i> <i>sentencia</i> } <i>alternativa</i>	<i>lista_sent</i> → ε
5	si id < num { <i>sentencia</i> } <i>alternativa</i>	<i>sentencia</i> → id = <i>expr</i> ;
6	si id < num { id = <i>expr</i> ; } <i>alternativa</i>	<i>expr</i> ⇒* dec (G7)
7	si id < num { id = dec ; } <i>alternativa</i>	<i>alternativa</i> → sino <i>bloque</i>
8	si id < num { id = dec ; } sino <i>bloque</i>	<i>bloque</i> → { <i>lista_sent</i> }
9	si id < num { id = dec ; } sino { <i>lista_sent</i> }	<i>lista_sent</i> → <i>lista_sent</i> <i>sentencia</i>
10	si id < num { id = dec ; } sino { <i>lista_sent</i> <i>sentencia</i> }	<i>lista_sent</i> → ε
11	si id < num { id = dec ; } sino { <i>sentencia</i> }	<i>sentencia</i> → id = <i>expr</i> ;
12	si id < num { id = dec ; } sino { id = <i>expr</i> ; }	<i>expr</i> ⇒* dec (G7)
13	si id < num { id = dec ; } sino { id = dec ; }	<i>fin</i>

Tabla B7

Derivaciones más a la izquierda de la gramática G7 (expresiones)

Paso	Forma sentencial	Producción aplicada
Ejemplo 7.1: vida - 25 < 10		
<i>Cadena de tokens: id - num < num</i>		
0	<i>expr</i>	<i>expr</i> → <i>conj</i>
1	<i>conj</i>	<i>conj</i> → <i>rel</i>
2	<i>rel</i>	<i>rel</i> → <i>arit</i> <i>oprel</i> <i>arit</i>
3	<i>arit</i> <i>oprel</i> <i>arit</i>	<i>arit</i> → <i>arit</i> - <i>term</i>
4	<i>arit</i> - <i>term</i> <i>oprel</i> <i>arit</i>	<i>arit</i> → <i>term</i>
5	<i>term</i> - <i>term</i> <i>oprel</i> <i>arit</i>	<i>term</i> → <i>unario</i>
6	<i>unario</i> - <i>term</i> <i>oprel</i> <i>arit</i>	<i>unario</i> → <i>factor</i>
7	<i>factor</i> - <i>term</i> <i>oprel</i> <i>arit</i>	<i>factor</i> → id
8	id - <i>term</i> <i>oprel</i> <i>arit</i>	<i>term</i> → <i>unario</i>
9	id - <i>unario</i> <i>oprel</i> <i>arit</i>	<i>unario</i> → <i>factor</i>
10	id - <i>factor</i> <i>oprel</i> <i>arit</i>	<i>factor</i> → num

Paso	Forma sentencial	Producción aplicada
11	id - num <i>oprel arit</i>	<i>oprel</i> → <
12	id - num < <i>arit</i>	<i>arit</i> → <i>term</i>
13	id - num < <i>term</i>	<i>term</i> → <i>unario</i>
14	id - num < <i>unario</i>	<i>unario</i> → <i>factor</i>
15	id - num < <i>factor</i>	<i>factor</i> → num
16	id - num < num	<i>fin</i>

Ejemplo 7.3: vida <= 0 || !vivo

Cadena de tokens: **id** <= **num** || **!** **id**

0	<i>expr</i>	<i>expr</i> → <i>expr</i> <i>conj</i>
1	<i>expr</i> <i>conj</i>	<i>expr</i> → <i>conj</i>
2	<i>conj</i> <i>conj</i>	<i>conj</i> → <i>rel</i>
3	<i>rel</i> <i>conj</i>	<i>rel</i> → <i>arit oprel arit</i>
4	<i>arit oprel arit</i> <i>conj</i>	<i>arit</i> → <i>term</i>
5	<i>term oprel arit</i> <i>conj</i>	<i>term</i> → <i>unario</i>
6	<i>unario oprel arit</i> <i>conj</i>	<i>unario</i> → <i>factor</i>
7	<i>factor oprel arit</i> <i>conj</i>	<i>factor</i> → id
8	id <i>oprel arit</i> <i>conj</i>	<i>oprel</i> → <=
9	id <= <i>arit</i> <i>conj</i>	<i>arit</i> → <i>term</i>
10	id <= <i>term</i> <i>conj</i>	<i>term</i> → <i>unario</i>
11	id <= <i>unario</i> <i>conj</i>	<i>unario</i> → <i>factor</i>
12	id <= <i>factor</i> <i>conj</i>	<i>factor</i> → num
13	id <= num <i>conj</i>	<i>conj</i> → <i>rel</i>
14	id <= num <i>rel</i>	<i>rel</i> → <i>arit</i>
15	id <= num <i>arit</i>	<i>arit</i> → <i>term</i>
16	id <= num <i>term</i>	<i>term</i> → <i>unario</i>
17	id <= num <i>unario</i>	<i>unario</i> → ! <i>unario</i>
18	id <= num ! <i>unario</i>	<i>unario</i> → <i>factor</i>
19	id <= num ! <i>factor</i>	<i>factor</i> → id
20	id <= num ! id	<i>fin</i>

Ejemplo 7.2: 2 * (velocidad + 1.5) / 4

Cadena de tokens: **num** * (**id** + **dec**) / **num**

0	<i>expr</i>	<i>expr</i> → <i>conj</i>
1	<i>conj</i>	<i>conj</i> → <i>rel</i>
2	<i>rel</i>	<i>rel</i> → <i>arit</i>
3	<i>arit</i>	<i>arit</i> → <i>term</i>
4	<i>term</i>	<i>term</i> → <i>term</i> / <i>unario</i>
5	<i>term</i> / <i>unario</i>	<i>term</i> → <i>term</i> * <i>unario</i>
6	<i>term</i> * <i>unario</i> / <i>unario</i>	<i>term</i> → <i>unario</i>
7	<i>unario</i> * <i>unario</i> / <i>unario</i>	<i>unario</i> → <i>factor</i>
8	<i>factor</i> * <i>unario</i> / <i>unario</i>	<i>factor</i> → num
9	num * <i>unario</i> / <i>unario</i>	<i>unario</i> → <i>factor</i>
10	num * <i>factor</i> / <i>unario</i>	<i>factor</i> → (<i>expr</i>)
11	num * (<i>expr</i>) / <i>unario</i>	<i>expr</i> → <i>conj</i>
12	num * (<i>conj</i>) / <i>unario</i>	<i>conj</i> → <i>rel</i>

Paso	Forma sentencial	Producción aplicada
13	num * (<i>rel</i>) / unario	<i>rel</i> → <i>arit</i>
14	num * (<i>arit</i>) / unario	<i>arit</i> → <i>arit</i> + <i>term</i>
15	num * (<i>arit</i> + <i>term</i>) / unario	<i>arit</i> → <i>term</i>
16	num * (<i>term</i> + <i>term</i>) / unario	<i>term</i> → unario
17	num * (unario + <i>term</i>) / unario	unario → <i>factor</i>
18	num * (<i>factor</i> + <i>term</i>) / unario	<i>factor</i> → id
19	num * (id + <i>term</i>) / unario	<i>term</i> → unario
20	num * (id + unario) / unario	unario → <i>factor</i>
21	num * (id + <i>factor</i>) / unario	<i>factor</i> → dec
22	num * (id + dec) / unario	unario → <i>factor</i>
23	num * (id + dec) / <i>factor</i>	<i>factor</i> → num
24	num * (id + dec) / num	<i>fin</i>

Ejemplo 7.4: municion > 0 && escudo != verdadero

Cadena de tokens: **id** > **num** && **id** != verdadero

0	<i>expr</i>	<i>expr</i> → <i>conj</i>
1	<i>conj</i>	<i>conj</i> → <i>conj</i> && <i>rel</i>
2	<i>conj</i> && <i>rel</i>	<i>conj</i> → <i>rel</i>
3	<i>rel</i> && <i>rel</i>	<i>rel</i> → <i>arit</i> oprel <i>arit</i>
4	<i>arit</i> oprel <i>arit</i> && <i>rel</i>	<i>arit</i> → <i>term</i>
5	<i>term</i> oprel <i>arit</i> && <i>rel</i>	<i>term</i> → unario
6	unario oprel <i>arit</i> && <i>rel</i>	unario → <i>factor</i>
7	<i>factor</i> oprel <i>arit</i> && <i>rel</i>	<i>factor</i> → id
8	id oprel <i>arit</i> && <i>rel</i>	oprel → >
9	id > <i>arit</i> && <i>rel</i>	<i>arit</i> → <i>term</i>
10	id > <i>term</i> && <i>rel</i>	<i>term</i> → unario
11	id > unario && <i>rel</i>	unario → <i>factor</i>
12	id > <i>factor</i> && <i>rel</i>	<i>factor</i> → num
13	id > num && <i>rel</i>	<i>rel</i> → <i>arit</i> oprel <i>arit</i>
14	id > num && <i>arit</i> oprel <i>arit</i>	<i>arit</i> → <i>term</i>
15	id > num && <i>term</i> oprel <i>arit</i>	<i>term</i> → unario
16	id > num && unario oprel <i>arit</i>	unario → <i>factor</i>
17	id > num && <i>factor</i> oprel <i>arit</i>	<i>factor</i> → id
18	id > num && id oprel <i>arit</i>	oprel → !=
19	id > num && id != <i>arit</i>	<i>arit</i> → <i>term</i>
20	id > num && id != <i>term</i>	<i>term</i> → unario
21	id > num && id != unario	unario → <i>factor</i>
22	id > num && id != <i>factor</i>	<i>factor</i> → verdadero
23	id > num && id != verdadero	<i>fin</i>

Apéndice C

Otros errores semánticos

Además de los errores E01 y E03 de la Tabla 5, el analizador semántico verifica los ocho errores de la Tabla C1. También advierte cuando un estado que no es final no tiene transiciones de salida, porque el personaje quedaría atrapado en él. No es un error porque puede ser intencional mientras se diseña.

Tabla C1

Otros errores semánticos que detecta el compilador

Código	Error	Regla que viola
E02	Evento no declarado, en una transición o en una simulación	Los símbolos de entrada pertenecen a Σ
E04	Estado inicial ausente o repetido	Existe exactamente un q_0
E05	Estado inalcanzable desde el estado inicial	Todo estado se alcanza desde q_0
E06	Nombre repetido o usado para dos cosas	Cada nombre identifica una sola cosa
E07	Atributo no declarado	Todo atributo usado está en la tabla de símbolos
E08	Tipos incompatibles en expresiones, guardas y asignaciones	Reglas de tipos del lenguaje
E09	Estado final con transiciones de salida	Un estado de F no tiene salidas
E10	Simulación de un personaje no declarado	La simulación usa un personaje existente

Apéndice D

Pruebas de los analizadores

Las pruebas están en la carpeta *pruebas* del repositorio del proyecto (<https://github.com/DavidRamPari/pixellang>). Cada tabla muestra la entrada y la salida del driver.

Analizador léxico

La Tabla D1 muestra los tokens que se obtienen de una entrada válida; el comentario no produce ninguno. La Tabla D2 muestra entradas con caracteres o lexemas que no pertenecen al léxico.

Tabla D1

Tokens de una entrada válida

Línea	Entrada y tokens
1	daño_golpe: entero = 25; ID DOS_PUNTOS ENTERO ASIGNA NUM_ENTERO PUNTO_COMA
2	vivo: lógico = verdadero; ID DOS_PUNTOS LOGICO ASIGNA VERDADERO PUNTO_COMA
3	cuando fin_aturdimiento si vida <= 0 -> Derrotado; // comentario CUANDO ID SI ID MENOR_IGUAL NUM_ENTERO FLECHA ID PUNTO_COMA

Tabla D2

Pruebas del analizador léxico con entradas inválidas

Entrada	Error reportado y explicación
vida: entero = 100 @ 2;	línea 1, columna 19: token recognition error at: '@' El carácter @ no pertenece a ningún patrón.
velocidad: real = 2.;	línea 1, columna 19: token recognition error at: '.' Falta un dígito después del punto: 2 es NUM_ENTERO y el punto queda solo.
eventos saltar, #atacar;	línea 1, columna 16: token recognition error at: '#' El carácter # no pertenece al léxico; atacar se reconoce como ID.
y: real = 3.14.15;	línea 1, columna 14: token recognition error at: '.' 3.14 es NUM_REAL y el segundo punto queda sin regla.
vivo: logico = verdadero & falso;	línea 1, columna 25: token recognition error at: '& ' El lexer intenta reconocer && y falla en el espacio.

Analizador sintáctico

Los 35 ejemplos del Apéndice A se analizan sin errores. La Tabla D3 muestra programas con errores sintácticos; los mensajes son los de ANTLR.

Tabla D3

Pruebas del analizador sintáctico

Prueba	Programa y salida del driver
S1	Falta el punto y coma al final de la transición. <pre>inicial estado Reposo { cuando recibir_golpe -> Aturdido }</pre> Error sintáctico en la línea 5, columna 4: missing ';' at '}'
S2	Falta la flecha -> entre el evento y el destino. <pre>inicial estado Reposo { cuando recibir_golpe Aturdido; }</pre> Error sintáctico en la línea 4, columna 29: mismatched input 'Aturdido' expecting {'si', '->'}
S3	La selección si está escrita sin llaves. <pre>al_entrar { si vida < 50 velocidad = 0.5; }</pre> Error sintáctico en la línea 6, columna 33: missing '{' at 'velocidad' Error sintáctico en la línea 7, columna 8: extraneous input 'cuando' expecting {'si', '}', ID}
S4	Falta la declaración de eventos. <pre>personaje Caballero { vida: entero = 100; inicial estado Reposo { }</pre> Error sintáctico en la línea 3, columna 4: extraneous input 'inicial' expecting {'eventos', ID} Error sintáctico en la línea 3, columna 26: mismatched input '{' expecting ':'
S5	Hay una transición fuera de un estado. <pre>eventos recibir_golpe; cuando recibir_golpe -> Aturdido; inicial estado Reposo { }</pre> Error sintáctico en la línea 3, columna 4: mismatched input 'cuando' expecting {'estado', 'inicial', 'final'}
S6	El estado no tiene nombre. <pre>inicial estado { cuando recibir_golpe -> Reposo; }</pre> Error sintáctico en la línea 3, columna 19: missing ID at '{'

Analizador semántico

Cada prueba de la Tabla D4 es un programa correcto en lo léxico y lo sintáctico que contiene uno de los diez errores semánticos.

Tabla D4

Pruebas del analizador semántico

Prueba	Programa y salida del driver
E01	La transición apunta a Volando, que no se declaró. <pre>estado Saltando { cuando aterrizar -> Reposo; cuando saltar -> Volando; }</pre> Error semántico E01 en la línea 6, columna 25: la transición de 'Saltando' con 'saltar' va hacia 'Volando', que no es un estado declarado
E02	Los eventos caer y agacharse no están declarados. <pre>estado Saltando { cuando caer -> Reposo; } simular Caballero: saltar, aterrizar, agacharse;</pre> Error semántico E02 en la línea 4, columna 29: el evento 'caer' no está declarado en 'Caballero' Error semántico E02 en la línea 6, columna 38: el evento 'agacharse' no está declarado en 'Caballero'
E03	La transición sin guarda está antes que la que tiene guarda. <pre>// estado Aturdido cuando recibir_golpe -> Aturdido; cuando fin_aturdimiento -> Reposo; cuando fin_aturdimiento si vida <= 0 -> Derrotado;</pre> Error semántico E03 en la línea 9, columna 8: en 'Aturdido', la transición con 'fin_aturdimiento' de la línea 9 nunca se toma: la de la línea 8 responde al mismo evento sin guarda
E04	El caballero no tiene estado inicial y el Slime tiene dos. <pre>// personaje Caballero estado Reposo { cuando saltar -> Saltando; } estado Saltando { cuando aterrizar -> Reposo; } // personaje Slime inicial estado Patrulla { inicial estado Aplastado {</pre> Error semántico E04 en la línea 1, columna 10: el personaje 'Caballero' no tiene estado inicial Error semántico E04 en la línea 11, columna 19: el personaje 'Slime' tiene más de un estado inicial: 'Patrulla', 'Aplastado'
E05	Ninguna transición lleva a Nadando. <pre>// Reposo: cuando saltar -> Saltando; estado Saltando { cuando aterrizar -> Reposo; } estado Nadando { cuando aterrizar -> Reposo; }</pre>

Prueba Programa y salida del driver

Error semántico E05 en la línea 5, columna 11:
el estado 'Nadando' no es alcanzable desde 'Reposo'

E06 Se repiten un atributo y un evento, y un estado usa el nombre de un evento.

```
vida: entero = 100;
vida: real = 50.0;
eventos saltar, recibir_golpe, recibir_golpe;
inicial estado Reposo { cuando saltar -> saltar; }
estado saltar { cuando recibir_golpe -> Reposo; }
```

Error semántico E06 en la línea 3, columna 4:
el atributo 'vida' ya fue declarado en la línea 2

Error semántico E06 en la línea 4, columna 35:
el evento 'recibir_golpe' ya fue declarado en la línea 4

Error semántico E06 en la línea 6, columna 11:
'saltar' ya es el nombre de un evento (línea 4); un nombre no puede usarse para dos cosas distintas

E07 escudo y energia no son atributos del personaje.

```
al_entrar { escudo = 10; }
cuando recibir_golpe si energia > 0 -> Reposo;
```

Error semántico E07 en la línea 5, columna 20:
'escudo' no es un atributo de 'Caballero'

Error semántico E07 en la línea 6, columna 32:
'energia' no es un atributo de 'Caballero'

E08 Tipos que no corresponden en una inicialización, una suma, una condición y una guarda.

```
vida: entero = 2.5;
    vida = vivo + 1;
    si vida { vivo = falso; }
    cuando recibir_golpe si vida -> Reposo;
```

Error semántico E08 en la línea 2, columna 4:
el atributo 'vida' es entero y se inicializa con un valor real

Error semántico E08 en la línea 7, columna 19:
el operador '+' necesita números y recibió logico

Error semántico E08 en la línea 8, columna 15:
la condición de 'si' debe ser logico, no entero

Error semántico E08 en la línea 10, columna 32:
la guarda de la transición debe ser logico, no entero

E09 Derrotado es final pero tiene una transición de salida.

```
final estado Derrotado {
    cuando revivir -> Reposo;
}
```

Error semántico E09 en la línea 6, columna 17:
'Derrotado' es un estado final y no puede tener transiciones

E10 La simulación se refiere a Mago, que no existe.

```
simular Mago: recibir_golpe;
```

Error semántico E10 en la línea 6, columna 8:
no existe un personaje llamado 'Mago'
